

Practical 3

AIM: To Implementation of max-heap sort algorithm

1 ALGORITHM:

Algorithm max-heap sort.
Input: An array A containing n elements.
Output: The array sorted in ascending order using max heap sort.
Step 1:- Start
Step 2:- Start with the last element not - last node
 $i = n/2 - 1$
2. for each node from i down to 0, call the heapify(i)
3. In heapify(i), compare the parent node with left right.
Step 3:- find the largest element among the parent left child right child.
2. If largest element is not parent swap the parent with largest child.
Repeat the heapify process for affected subtree.
Step 4:- set $i = n - 1$
2. swap the root element $A[0]$ with $A[i]$
Step 5:- Reduce the heap size by 1.
Step 6:- Apply heapify(i) to the root node(i) to restore the max heap property.
Step 7:- decrease i by 1
Step 8:- repeat step 4-7 until $i = 0$.
Step 9:- Stop.

2 PROGRAM:

```
#include <iostream>

using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
```

```
if (left < n && arr[left] > arr[largest])
largest = left;
if (right < n && arr[right] > arr[largest])
largest = right;
if (largest != i){
int temp = arr[i];
arr[i] = arr[largest];
arr[largest] = temp;
heapify(arr, n, largest);
}}
void heapSort(int arr[], int n){
// Build Max Heap
for (int i= n/2-1;i >= 0;i--){
heapify(arr, n, i);
// Heap Sort
for (int i=n-1;i>0;i--){
int temp = arr[0];
arr[0] = arr[i];
arr[i] = temp;
heapify(arr, i, 0);}}
int main(){
int arr[] = {12, 11, 13, 5, 6, 7};
int n = 6;
cout << "Original array: ";
for (int i = 0; i < n; i++)
cout << arr[i] << " ";
heapSort(arr, n);
cout << "\nSorted array: ";
for (int i = 0; i < n; i++)
cout << arr[i] << " ";
return 0;}
```

3 OUTPUT:

```
C:\WINDOWS\system32\cmd. x + v
Original array: 12 11 13 5 6 7
Sorted array: 5 6 7 11 12 13
C:\Users\YASWANTH\DAAG>
```

4 TIME COMPLEXITY:

Time complexity :-
At each step recursive call we move down one level in the heap.
A binary heap has approximately half the elements in the subtree we contain into.
So the recurrence is $T(n) = T(n/2) + O(1)$
By applying master theorem:- $T(n) = aT(n/b) + f(n)$
compare $T(n) = T(n/2) + 1$
 $a=1, b=2, k=0, p=0$
 $f(n) = 1 = O(1)$
By calculating $n \log_b a$
 $= n \log_2 1$
 $(\because \log_2 1 = 0)$
 $n^0 = 1 \Rightarrow f(n) = O(n \log_b a)$
So by master theorem case 2:- $T(n) = O(\log n)$
Heapify = $O(\log n)$.
So heap sort has n heapify operations:-
 $T(n) = n \times O(\log n)$
 $T(n) = O(n \log n)$.
For max-heap sort:-
Best & Average = Worst = $O(n \log n)$.
Worst case:-
 $T_{\text{worst}}(n) = T(n/2) + O(1)$
 $a=1, b=2, f(n)=1$
 $n \log_b a = 1$
 $T_{\text{worst}}(n) = O(n \log n)$
Average case:-
 $T_{\text{avg}}(n) = T(n/2) + O(1)$
 $a=1, b=2, f(n)=1$
 $n \log_b a = 1$
 $T_{\text{avg}}(n) = O(n \log n)$

5 Conclusion:

The Max-Heap Sort algorithm was successfully implemented and executed. The given elements were sorted in ascending order, and the time complexity was analyzed for the best, average, and worst cases. Thus, the implementation of the Max-Heap Sort algorithm was successfully done.